

SDOC Hackathon

Shipping Document Verification Pipeline

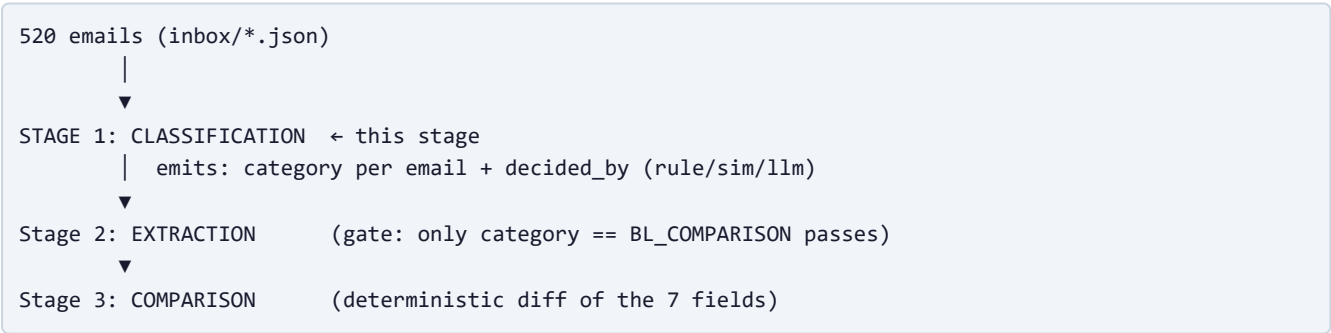
Architecture — Stage 1: Classification · Stage 2: Extraction · Stage 3: Comparison

2026-09-19 | Companion to DECISIONS.md

Stage 1 — Classification

Status: **DECIDED** (agreed 2026-09-19). This is the finalized architecture for the classification stage. Decision log: DECISIONS.md (D1–D9). Scoring weights: stage1 0.30 / stage3 0.20 / end-to-end 0.50.

Placement in the pipeline



Stage contract

Inbound: raw email record {email_id, from, subject, body, attachments[]} — filenames only, attachments are never opened here (D9).

Outbound (per email):

```
{"category": "BL_COMPARISON", "decided_by": "rule"}
```

- Categories (exact scorer vocabulary, `scoring.py:23`): `BL_COMPARISON` | `SI_REQUEST` | `INVOICE_QUERY` | `GENERAL` | `SPAM`
- `decided_by` is an internal extra — the scorer reads it for `rule_pct` for free (`scoring.py:55-59`); keep it in the submission payload.
- Never emits match/mismatch decisions — that belongs to later stages.

Architecture: cascade classifier (D1, D2)

No trained ML model (520 samples, 5 classes, imbalanced → overfit risk). Four-stage cost funnel; ~85–90% resolved free, LLM sees ~2–6%.

```
email (from, subject, body, attachment filenames)
|
— SIGNAL EXTRACTION (every email, free) —
|
signals = { has_SI, has_BL, n_att, is_reply, sender_domain,
            intent_check, intent_create, billing_terms, spam_score }
|
▼
STAGE 0: HARD RULES (precision ~100%, decided_by="rule")
  spam_score high OR sketchy domain      → SPAM
  has_SI AND has_BL AND intent_check     → BL_COMPARISON
  SI only + check intent                  → BL_COMPARISON
    ↳ flag: missing_attachment (feeds Stage 2 review)
  billing terms, no doc intent            → INVOICE_QUERY
  unresolved → Stage 1 (~50-100 left)
|
▼
STAGE 1: WEIGHTED EVIDENCE SCORE
  score[c] = Σ (signal × weight) per class
  e.g. BL_COMPARISON = 3·(SI∧BL) + 2·intent_check
       SI_REQUEST    = 2·intent_create - (SI∧BL)
       SPAM           = 2·spam_score + domain_rep
  Decide iff: top ≥ ACCEPT AND margin = top-2nd ≥ MARGIN
              AND no conflict (check+create both fired)
  else → Stage 2 (~30-80 left)
|
▼
STAGE 2: SEMANTIC SIMILARITY (tiebreaker, feeds back into Stage-1 scores)
  TF-IDF cosine: body vs 5 template centroids
  (centroids = 2-3 real body phrasings per class, from ~10 hand-read emails)
  Decide iff: best sim ≥ SIM_THRESHOLD and gap clear
              and no contradiction with evidence
  else → Stage 3 (~10-30 left)
|
▼
STAGE 3: GLM-5.3-FLASH API (temp 0, JSON out, D4)
  5-shot prompt (one real example per class) + email
  → {"category", "confidence", "reason"}
  confidence ≥ 0.85 → decide, decided_by="llm"
  confidence < 0.85 → status=NEEDS_REVIEW (D7: never guess)
```

Core principle (D3)

Classification uses **email metadata only**: subject, body, attachment filenames. Subjects are adversarial; body intent + attachment structure carry the signal.

Prompt contract (Stage 3 LLM)

- Categories: exact scorer vocabulary above
- Definitions must emphasize: trust **body intent** over subject (misleading subjects are planted); `RE_` replies with no attachments can still be `BL_COMPARISON`
- Output: `{"category": "...", "confidence": 0.0-1.0, "reason": "one sentence"}`
- One client (GLM-5.3-flash) shared with extraction fallback; whole dataset $\approx$ \$0.10–0.50

Calibration loop (free)

1. Run cascade with thresholds wide open
2. `POST /submit` → read stage-1 macro-F1 + `rule_pct`
3. Tighten `ACCEPT` / `MARGIN` / `SIM_THRESHOLD` until rules resolve max emails without accuracy drop
4. Repeat — each submission costs nothing

Verified dataset stats (computed from `ground_truth.json`, seed 42)

Category	Count	Notes
BL_COMPARISON	220	200 main set + 20 edge cases (email_501–520)
SI_REQUEST	125	bodies embed shipment details (irrelevant — exit here)
INVOICE_QUERY	75	
GENERAL	60	includes RPA bot notices, HR
SPAM	40	prize/phishing, sender domains are the giveaway

- 46 defect emails (all in main set) — the end-to-end denominator
- All 20 `NEEDS_REVIEW` are email_501–520 (edge cases); main set has zero
- Baseline all-GENERAL submission = 0.0124 final; perfect classification alone caps at 0.30

Traps owned by this stage

Trap	Handling
Misleading subjects (email_004/052 "REQUEST BL DRAFT" subject, comparison body; 79/124 comparisons lack "CONFIRM DOCS")	Body-intent signals, never subject keywords
Reply threads, no attachments (43 <code>RE_</code> + <code>CONFIRM DOCS</code> , 23 with zero attachments)	Escalate on absence; classify by intent
Missing attachment (email_507 SI-only, email_495 none)	classify <code>BL_COMPARISON</code> + <code>missing_attachment</code> flag
Silent submission default (missing email scores as GENERAL)	Assert <code>len(submission)==520</code> before every submit
LLM output fragility (fenced/truncated JSON)	JSON-extraction regex + 1 retry + fallback <code>NEEDS_REVIEW</code>
Crash mid-run loses progress	Checkpoint results per email; resumable
Escalation over-trigger tanks <code>escalation_precision</code>	Tune 0.85 threshold via free submissions

Stage 2 — Extraction

Status: **DECIDED** (2026-09-19). Finalized architecture for the extraction stage. Upstream: Stage 1 [Classification.md](#). Downstream: Stage 3 Comparison (deterministic). Problem-statement anchor: Capability 2 — "Extract data: for comparison requests, read the SI and BL attachments and identify the corresponding shipment fields."

Placement in the pipeline

```
Stage 1: CLASSIFICATION
|   gate: only category == BL_COMPARISON passes (~125 of 520)
▼
STAGE 2: EXTRACTION ← this stage
|   emits: 7-field record pair (SI + BL) or a pre-escalated review case
▼
Stage 3: COMPARISON (deterministic diff; only receives READY records)
```

What this stage is

Open the SI attachment → build a 7-field record. Open the BL attachment → build a 7-field record. Hand both to comparison.

- The email contributes nothing here — its job (gate + pointing at `attachments[]`) was done in Stage 1. All 7 fields live inside the attachment files.
- The email body is prose; it never lists the 7 fields for BL_COMPARISON. Ground truth also contains no field values (only `category/status/defect_fields/has_defect`) — nobody hands us values; extraction produces them.
- Extraction is a **pure reader**: never decides match/mismatch, never sees ground truth, never calls the scorer.

Stage contract

Inbound: email record (from Stage 1 gate) → `attachments[]` paths.

Outbound (per email):

```
{
  "si_doc": {"shipper": "...", "consignee": "...", "notify_party": "...",
            "port_of_loading": "...", "port_of_discharge": "...",
            "container_count": 6, "gross_weight_kg": 122640.0},
  "bl_doc": {same shape},
  "doc_status": "READY",
  "review_reason": null,
  "evidence": {"field": {"value": ..., "raw": "...", "label_found": "Gross Wt (kgs)",
                        "source_format": "txt"}}
```

- `evidence` is internal-only (demo/review UI): what was read, from where, under which label.
- Field types: parties/ports = `str|null`; `container_count` = `int|null`; `gross_weight_kg` = `float|null`.

- **null semantics are contractual:** null = field genuinely absent/blank in the source (drives `missing_value`), never a parse failure — parse failure is `readable:false` (drives `unreadable`).
- Non-READY exits leave the stage pre-escalated (D7: escalate, never guess). Stage 3 only sees `READY`.

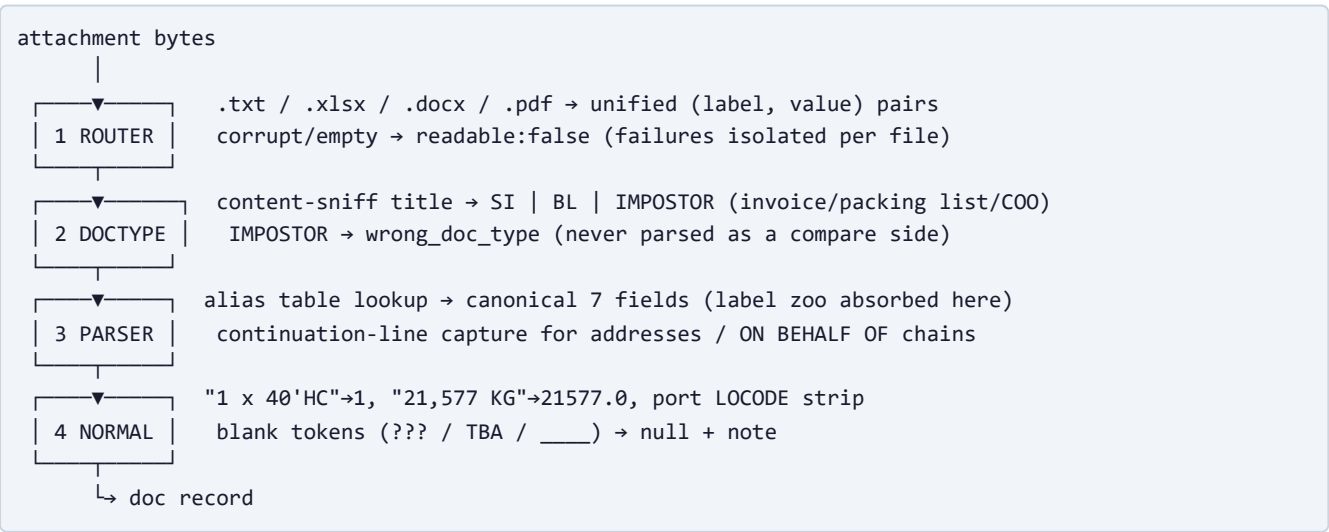
Exit statuses

doc_status	review_reason	Trigger	Where it comes from in the data
READY	null	both docs parsed, 7 fields present	~110 main-set pairs
NEEDS_REVIEW	missing_attachment	0 attachments, or SI only (decided before any file I/O)	email_506/508/510 (zero), 507/509 (SI-only)
NEEDS_REVIEW	wrong_doc_type	content-sniff finds an impostor title	email_501–505 ("BL" is a Commercial Invoice / Packing List / COO)
NEEDS_REVIEW	unreadable	file corrupt/empty/has no text layer	email_511_BL, 515_BL (garbled PDFs), 512–514 (image-only)
NEEDS_REVIEW	missing_value	parsed fine but ≥1 required field is blank in source	email_516–520 (??? / _____ / TBA / blank)

Document inventory (verified, seed 42 — 250 docs)

Type	Count	Files
.txt	~78%	most pairs + all edge docs
.pdf (text layer)	10 pairs	059, 160, 208, 273, 313, 351, 407, 411, 434, 499
.pdf (image-only)	3 pairs	512, 513, 514
.pdf (corrupt)	2	511_BL, 515_BL (garbled, no %EOF)
.docx	8 (all BLs)	055, 097, 107, 291, 302, 354, 435, 462
.xlsx	13 pairs	005, 055, 097, 107, 171, 243, 291, 300, 302, 354, 398, 462, 481, 496
no attachment	5	506, 508, 510 (zero) · 507, 509 (SI only)

Internal architecture — 4 sub-components



Core identification trick (same for every format): 1. Reduce the file to (label, value) pairs — colon-split for .txt, cell-adjacency for .xlsx/.docx, baseline-position for .pdf 2. Look the label up in one shared alias dictionary → canonical field name 3. Normalize the value per field type

The alias table is **data, not code** — one mapping consumed by all four adapters, so adding a format never forks the field logic. No pure-paragraph attachments exist; every format encodes label:value as colon, cell border, or pixel position.

1. Router (per-format adapters)

Ext	Adapter	Gotchas
.txt	read utf-8, errors=replace → line regex ^([^\:]{2,40}):\s*(.*)\$; continuation lines absorbed into previous value	win32 charmap crash is real: email_519_SI.txt contains 毛重
.xlsx	openpyxl: col A = label, col B = value	weight cell is a raw int , not a string; skip (BL INSTRUCTION, so_number) noise rows
.docx	python-docx table rows: cell[0] label, cell[1] value	labels carry bilingual suffixes (Shipper (发货人)); value cell contains name\naddr\naddr → first non-empty line is the company identity
.pdf	pymupdf spans: label at x≈20mm, value = spans right of it sharing baseline y	see weight/count trap below
corrupt/empty	per-file try/except → readable:false; zero-byte check first	sibling doc unaffected; email still exits NEEDS_REVIEW

2. Doc-type sniff (before parsing, all formats)

First-line title → SHIPPING INSTRUCTION | BILL OF LADING | else (COMMERCIAL INVOICE / PACKING LIST / CERTIFICATE OF ORIGIN) → wrong_doc_type . The generator stamps *** THIS IS A COMMERCIAL INVOICE - NOT A SHIPPING INSTRUCTION *** in impostor files.

3. Alias dictionary (the label zoo)

Built from pools.py:175-188 — the generator's exact source. Normalization before lookup: **uppercase** → **strip parentheticals (...)** and **(...)** **(CJK)** → **strip punctuation** → **collapse spaces**. So Gross Weight毛重(KGS) → GROSS WEIGHT , Shipper (发货人) → SHIPPER .

Canonical field	Aliases absorbed
shipper	Shipper · Shipper/Exporter · Shipper (Principal or Seller) · SHIPPER · 发货人
consignee	Consignee · Consignee (Non-Negotiable) · To the Order of · 收货人
notify_party	Notify Party · Notify · Notify Party/Intermediate Consignee · NOTIFY PARTY · 通知人
port_of_loading	Port of Loading · Port of Loading (POL) · Load Port · POL · 装货港
port_of_discharge	Port of Discharge · (POD) · Discharge Port · POD · 卸货港
container_count	No. of Containers · Total Containers · No. of Containers or Packages · Container Count · 箱数
gross_weight_kg	Gross Weight (KG) · Gross Wt (kgs) · Gross Weight毛重(KGS) · GROSS WEIGHT · 毛重 KGS

Longest-alias-first matching is not optional — Notify Party/Intermediate Consignee contains "Consignee"; naive matching routes notify values into consignee. Match exact against normalized full label, longest first.

4. Typed value normalizers

Field	Rule
parties	first line = company identity; ON BEHALF OF chains absorbed as evidence (email_520)
ports	strip trailing (MYPKG) -style UN/LOCODE → clean name; keep code as validation evidence
container_count	(\d+)\s*x → int; bare int accepted
gross_weight_kg	strip commas, require KG/KGS on the line → float; LBS or missing unit → null + note, never guessed
all	blank tokens ??? / _____ / TBA / TBC / N/A / ____MT / empty → null + note (drives missing_value)

Disambiguation guards (the traps)

- **PDF weight/count trap:** GROSS WEIGHT (KG) appears twice — as a container-table column header (per-row values) and as the TOTAL ...: line. Anchor **count** on the No. of Containers: N x size summary line and **weight** on the TOTAL line only; ignore table-body rows.
- **Label not found anywhere** → null + note label not found → missing_value (correct escalation, not a mismatch)
- **Two conflicting lines match same field** → prefer non-blank, record both in evidence
- **Sanity cross-check:** weight/count ratio outside ~18–26 MT/container → flag in evidence (plausibility only, doesn't block)

Architectural properties

- **Single canonical field schema** — all four adapters produce identical records via the shared alias table
- **Per-file isolation** — exception in one attachment → that doc unreadable, sibling unaffected, email still exits NEEDS_REVIEW rather than crashing the batch (try/except everywhere)
- **Determinism** — no LLM in this stage's main path (D6); LLM fallback is an optional side-door behind a flag for docs deterministic parsing fails on
- **OCR gate** — image-only PDFs: detect via page.get_text() empty + embedded image present. GLM-OCR (0.9B, Docker/vLLM, D5) is a future sub-component plugged into Router slot 1; OCR output feeds the .txt parser unchanged. If OCR jumbles a line, that field exits null and escalates — never guessed. (D9: OCR lives here, not in Stage 1.)
- **Checkpointing** at the stage boundary — extracted records cached to disk keyed by email_id + file hash; re-runs don't re-parse
- **UTF-8 everywhere** — open(..., encoding='utf-8', errors='replace') globally

Decision log (this stage)

#	Decision	Rationale
E1	Fields extracted from attachments only; email body/subject not used here	The 7 fields exist only inside the documents
E2	Alias table as data, shared across adapters	Generator's synonym zoo; one source of truth
E3	null = genuinely absent; readable:false = parse failure	Separates missing_value from unreadable — the two scored review reasons
E4	Escalated docs exit Stage 2 pre-classified with review_reason	D7; Stage 3 never guesses from partial data
E5	Scanned PDFs (512–514): OCR for evidence, still exit unreadable	Ground truth marks them NEEDS_REVIEW — reporting a clean compare would LOSE reliability points; OCR text attached as demo evidence for judges

#	Decision	Rationale
E6	Impostor sniff precedes parsing	Impostor docs must never be parsed as a compare side
E7	Per-file try/except isolation	One bad doc never kills the email or the batch

Verification harness (free, no ground-truth values needed)

`ground_truth.json` tells us *which* fields differ. So extraction is checkable **without knowing actual values**:

- For `has_defect=false` emails: all 7 extracted SI/BL pairs must **agree** → any disagreement = parser bug
- On `defect_fields`: extracted pairs must **differ** → agreement = parser bug
- On NEEDS_REVIEW edges: the expected field must be null/unreadable, review_reason must match the table above
- Sanity: weight/count ratio in 18–26 MT range across all pairs

Mismatch between agreement-structure and GT = a parser bug with the exact email_id + field named.

Build steps (ordered)

1. `pip install python-docx openpyxl pymupdf` + global utf-8 read helper
2. Router + .txt parser + alias table (~78% of docs) → run harness
3. .xlsx + .docx parsers → harness
4. .pdf text-layer path (pymupdf) → harness
5. Doc-type sniff + blank detection → edges 501–505, 516–520 clean
6. Corrupt/empty + missing-attachment exits → edges 506–511, 515 handled
7. GLM-OCR Docker for scanned PDFs (E5) → last

Stage 3 — Comparison

Status: **DECIDED** (2026-09-19). Finalized architecture for the comparison stage. Upstream: Stage 2 Extraction.md (READY record pairs only). This stage produces the final submission fields. Problem-statement anchor: Capability 3 — *"Compare: check the values and surface any mismatched fields, showing the SI and BL values side by side."* — and the headline example: SI 3 containers / BL 4 → flag only container_count, show SI: 3 / BL: 4.

Placement in the pipeline

```
Stage 2: EXTRACTION
  | gate: only doc_status == READY record pairs (~110 emails)
  ▼
STAGE 3: COMPARISON ← this stage
  | emits: has_defect, defect_fields, side-by-side evidence
  ▼
Submission assembly (520 emails: Stage 1 categories + Stage 2 reviews + Stage 3 verdicts)
```

What this stage is

Pure deterministic code (D6 — **never LLM**). Diff the two 7-field records, emit which fields differ. Reproducible, explainable, debuggable — every verdict traces to two values a human can eyeball.

It carries **0.50 of the final score** (end-to-end weight) despite being the smallest stage: e2e success requires category == BL_COMPARISON **AND** has_defect **AND** defect_fields matching ground truth **exactly** (scoring.py:159). One false-flagged field on an otherwise-correct email = zero e2e credit for that email. Precision on the field set is everything.

Stage contract

Inbound (per email, from Stage 2):

```
{"si_doc": {7 fields}, "bl_doc": {7 fields}, "evidence": {...}}
```

Outbound:

```
{
  "has_defect": true,
  "defect_fields": ["container_count", "gross_weight_kg"],
  "diff_report": {"container_count": {"si": 6, "bl": 5},
                  "gross_weight_kg": {"si": 21577.0, "bl": 21577.0, ...}}
}
```

- diff_report is internal (demo artifact / review UI): the "SI: X / BL: Y" side-by-side the problem statement asks for.
- Non-READY emails never arrive here — Stage 2 exits them pre-escalated (missing_attachment / wrong_doc_type / unreadable / missing_value).

Equality semantics per field (the one real design decision)

The scorer demands the exact defect-field set. Too strict → false alarms (stage3 precision + e2e loss); too loose → missed defects (stage3 recall + e2e loss). Empirically verified against all 46 defect emails and OK pairs:

Field	Equality rule	Verified rationale (from data)
shipper / consignee / notify_party	normalize: uppercase → collapse whitespace → strip punctuation → exact string equality	Mutations are <i>entirely different companies</i> (e.g. SI "EAST BRIGHT FZ-LLC" vs BL "UAB NOVAKOPA") — no fuzzy matching needed; case/punct-insensitivity only absorbs formatting noise
port_of_loading / port_of_discharge	normalize: uppercase → strip punct → collapse ws → exact equality on the name part (LOCODE parenthetical removed before compare)	Critical finding: in all 16 txt port-defect pairs the LOCODE stays <i>identical</i> while the name mutates (SI MOMBASA, KENYA (KEMBA) vs BL TUTICORIN, INDIA (KEMBA)). The generator mutates only the name field, never pod_code . Comparing LOCODEs would miss 100% of port defects; comparing names catches all. LOCODE is kept as evidence only
container_count	exact int equality	Deltas are ±1, ±2 — no fuzz
gross_weight_kg	exact numeric equality after comma-strip	Deltas are ±500/±1000/±2000 — clean integers, no float tolerance needed; on OK pairs the raw weight strings were byte-identical (51/51 checked)

Normalization pipeline (both sides, both field kinds): upper() → strip punctuation → collapse whitespace (alias tables for ports live in Stage 2's normalizer; comparison receives canonical values).

Algorithm

```
defect_fields = []
for f in [shipper, consignee, notify_party, port_of_loading,
         port_of_discharge, container_count, gross_weight_kg]:
    si_v, bl_v = normalize(si_doc[f]), normalize(bl_doc[f])
    if si_v is None or bl_v is None:      # defensively – Stage 2 should have caught it
        exit to NEEDS_REVIEW/missing_value (never guess)
    if not equal(f, si_v, bl_v):
        defect_fields.append(f)
        diff_report[f] = {si: si_v, bl: bl_v}

has_defect = bool(defect_fields)
if not defect_fields: report = "No mismatch detected"
```

ON BEHALF OF chains: comparison's final say on "company identity" — Stage 2 supplies the first-line identity as the value; if equality fails on identity but the chain contains the counterpart (evidence), flag for review rather than defect (verified trap: email_520-style).

Verified dataset facts this stage must handle

Fact	Numbers	Implication
Defect email composition	20 single-field, 26 two-field (46 total, all main set)	Two-field pairs are common — partial detection scores zero on e2e
Most-common defect combos	container_count ×5 alone; (container_count, port_of_discharge) ×4; (container_count, gross_weight_kg) ×4	container_count is in ~59% of defect emails (19/32 txt+binary)

Fact	Numbers	Implication
Defect field frequency	container_count 19, port_of_discharge 13, gross_weight_kg 12, notify_party 8, consignee 7, shipper 7, port_of_loading 6	
Label synonymy on OK pairs	consignee label differs SI-vs-BL in ~53% of pairs (10/19 sampled)	Synonymy is Stage 2's job; by the time values reach Stage 3, labels are gone — only values diff
Same-value-different-format	0 occurrences of differently-formatted identical weights on OK pairs	No fuzzy-tolerance arms race needed; clean equality is correct

Exit paths from this stage

Outcome	Emitted	Goes to
All 7 equal	has_defect=false, defect_fields=[], "No mismatch detected"	submission (status OK)
≥1 differ	has_defect=true, defect_fields=[...] + diff_report	submission (status MISMATCH)
Defensive: null reached here	route back → NEEDS_REVIEW/missing_value	review queue (never a guess, D7)

Architectural properties

- **Pure function** — record pair in, verdict out; no I/O, no state, fully unit-testable
- **Determinism** (D6) — same input, same verdict, always; every flag traceable to two printed values
- **Zero cost** — no API, no GPU; 46 defect + ~64 clean emails diff in milliseconds
- **Evidence-first demo artifact** — diff_report doubles as the judge-facing "which email, what mismatched, SI vs BL side by side" report the problem statement asks for
- **Exact-set discipline** — defect_fields is emitted as the complete set, never partial; if any field's parse is doubtful, the email exits to review rather than emitting a wrong set (e2e is binary per email)

Decision log (this stage)

#	Decision	Rationale
C1	Compare normalized <i>name parts</i> of ports, never LOCODE	LOCODE stays constant across defects (16/16 verified) — comparing it would miss every port defect
C2	Exact numeric equality for weight/count; exact normalized string for parties/ports	Mutation deltas are large (±500 kg, ±1 ctr, whole-company swaps); fuzz only adds false alarms
C3	Punctuation/case/whitespace-insensitive normalization before equality	Zero same-value-different-format cases in data, but formatting noise absorption is free insurance
C4	Any null arriving at Stage 3 → escalate, never compare	D7; null-vs-value is not a diff, it's uncertainty
C5	diff_report side-by-side emitted for every email (clean or not)	Problem statement's example report format; doubles as demo evidence
C6	ON BEHALF OF identity mismatch w/ chain evidence → review, not defect	email_520 trap; identity ≠ textual inequality